

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 3010

OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 3010

OPERATING SYSTEM

WEEK 9 LECTURE1

SOCKET PROGRAMMING

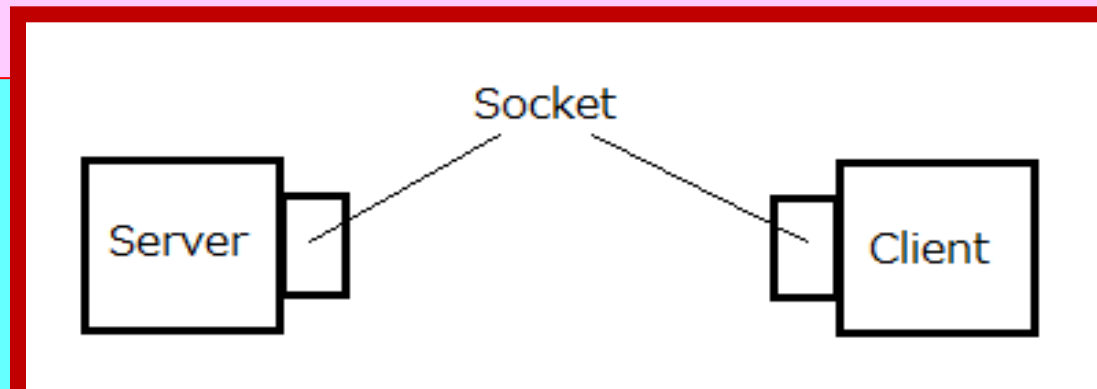
WISH YOU ALL THE BEST

INTER PROCESS OR NODE COMMUNICATION

- Most interprocess communication uses the *client server model*.
- These terms refer to the two processes which will be communicating with each other.
- One of the two processes, the *client*, connects to the other process, the *server*, typically to make a request for information.
- A good analogy is a person who makes a phone call to another person.
- Notice that the client needs to know of the existence of and the address of the server, but the server does not need to know the address of (or even the existence of) the client prior to the connection being established.
- Notice also that once a connection is established, both sides can send and receive information.

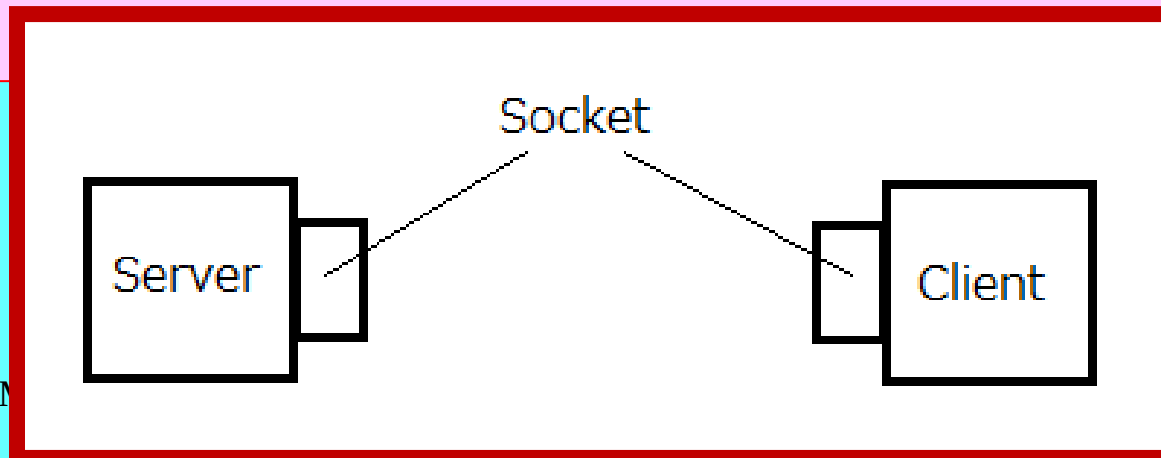
INTER PROCESS OR NODE COMMUNICATION

- The system calls for establishing a connection are somewhat different for the client and the server, but both involve the basic construct of a **socket**.
- A socket is one end of an interprocess communication channel.
- The two processes each establish their own socket.
- Imagine a socket as a seaport that allows a ship to unload and gather shipping, whereas socket is the place where a computer gathers and puts data into the internet



INTER PROCESS OR NODE COMMUNICATION

- A Socket is an end point of communication between two systems on a network.
- A socket is a combination of IP address and port on one system.
- On each system a socket exists for a process interacting with the socket on other system over the network.
- A combination of local socket and the socket at the remote system is also known a 'Four tuple' or '4-tuple'.
- Each connection between two processes running at different systems can be uniquely identified through their 4-tuple



INTER PROCESS OR NODE COMMUNICATION

- **Socket programming is a way of connecting two nodes on a network to communicate with each other.**
- **One socket(node) listens on a particular port at an IP, while other socket reaches out to the other to form a connection.**
- **Server forms the listener socket while client reaches out to the server.**

SOCKET PROGRAMMING

❑ TCP/IP (Transmission Control Protocol/Internet Protocol)

- TCP/IP, or the Transmission Control Protocol/Internet Protocol, is a suite of communication protocols used to interconnect network devices on the internet.
- TCP/IP can also be used as a communications protocol in a private network (an intranet or an extranet)
- The entire internet protocol suite -- a set of rules and procedures -- is commonly referred to as TCP/IP, though others are included in the suite.

SOCKET PROGRAMMING

□ TCP/IP (Transmission Control Protocol/Internet Protocol)

- **TCP/IP specifies how data is exchanged over the internet by providing end-to-end communications that identify how it should be broken into packets, addressed, transmitted, routed and received at the destination.**
- TCP/IP requires little central management, and it is designed to make networks reliable, with the ability to recover automatically from the failure of any device on the network.

SOCKET PROGRAMMING

❑ TCP/IP (Transmission Control Protocol/Internet Protocol)

- The two main protocols TCP and IP in the internet protocol suite serve specific functions.
- TCP defines how applications can create channels of communication across a network.
- It also manages how a message is assembled into smaller packets before they are then transmitted over the internet and reassembled in the right order at the destination address.
- IP defines how to address and route each packet to make sure it reaches the right destination.
- Each gateway computer on the network checks this IP address to determine where to forward the message.

SOCKET PROGRAMMING

❑ OSI MODEL

- The OSI Model (Open Systems Interconnection Model) is a conceptual framework used to describe the functions of a networking system.
- In the OSI reference model, the communications between a computing system are split into seven different abstraction layers: Physical, Data Link, Network, Transport, Session, Presentation, and Application.

7	Application Layer	Human-computer interaction layer, where applications can access the network services
6	Presentation Layer	Ensures that data is in a usable format and is where data encryption occurs
5	Session Layer	Maintains connections and is responsible for controlling ports and sessions
4	Transport Layer	Transmits data using transmission protocols including TCP and UDP
3	Network Layer	Decides which physical path the data will take
2	Data Link Layer	Defines the format of data on the network
1	Physical Layer	Transmits raw bit stream over the physical medium

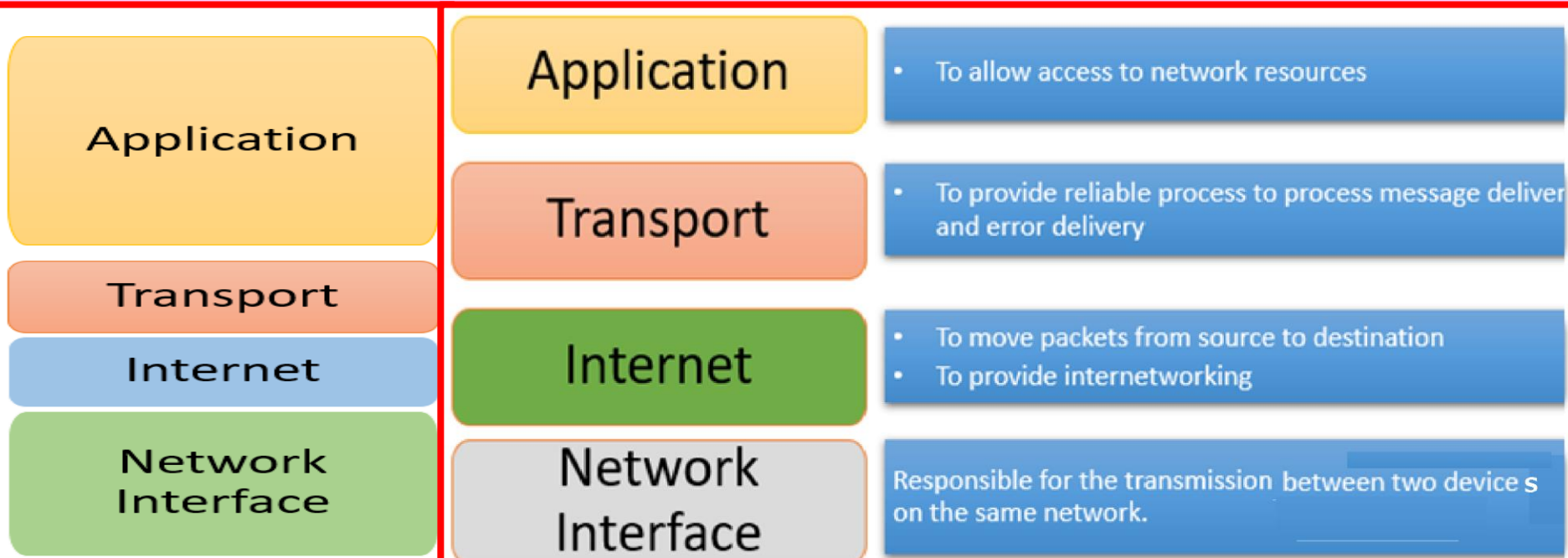
7 Layers of the OSI Model

Application	• End User layer • HTTP, FTP, IRC, SSH, DNS
Presentation	• Syntax layer • SSL, SSH, IMAP, FTP, MPEG, JPEG
Session	• Synch & send to port • API's, Sockets, WinSock
Transport	• End-to-end connections • TCP, UDP
Network	• Packets • IP, ICMP, IPSec, IGMP
Data Link	• Frames • Ethernet, PPP, Switch, Bridge
Physical	• Physical structure • Coax, Fiber, Wireless, Hubs, Repeaters

SOCKET PROGRAMMING

❑ TCP/IP MODEL

- TCP/IP Model helps you to determine how a specific computer should be connected to the internet and how data should be transmitted between them.
- The purpose of TCP/IP model is to allow communication over large distances.
- TCP/IP stands for Transmission Control Protocol/ Internet Protocol. TCP/IP Protocol Stack is specifically designed as a model to offer highly reliable and end-to-end byte stream over an unreliable internetwork.
- Application layer
- Transport layer
- An internet layer
- Network Interface Layer



SOCKET PROGRAMMING

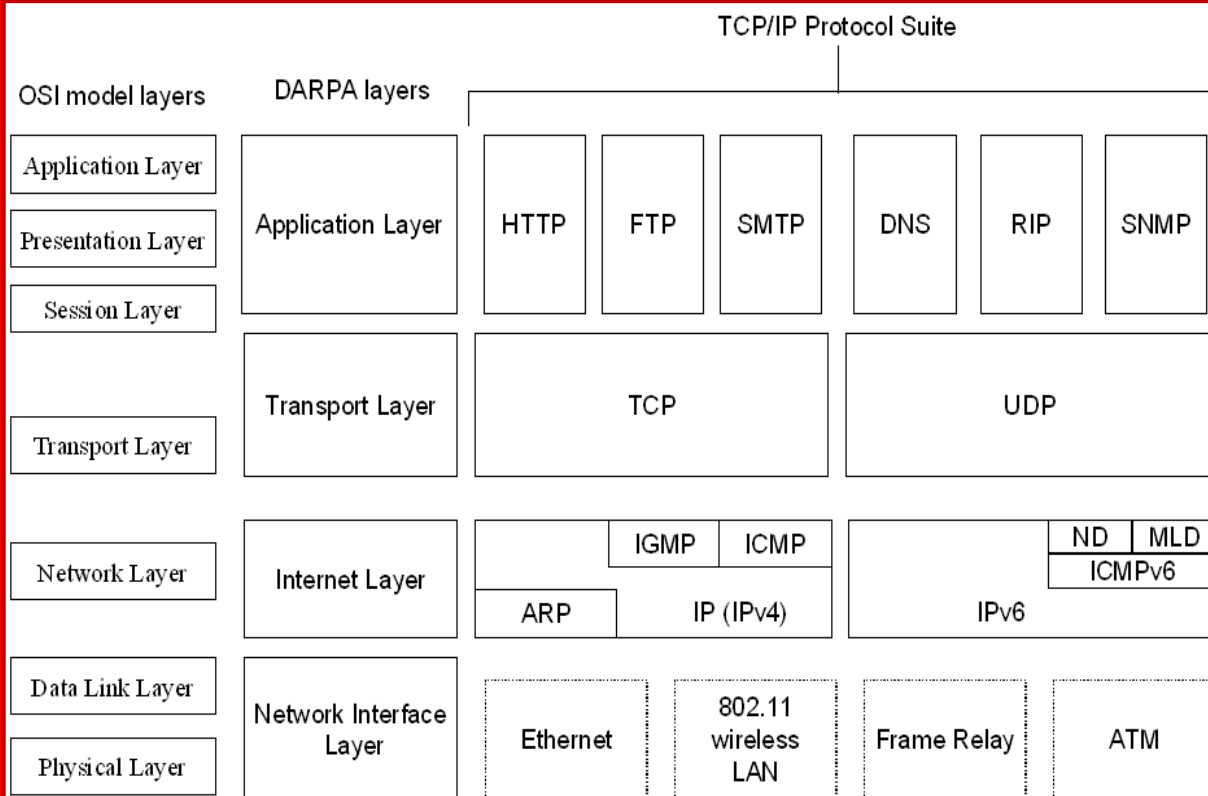
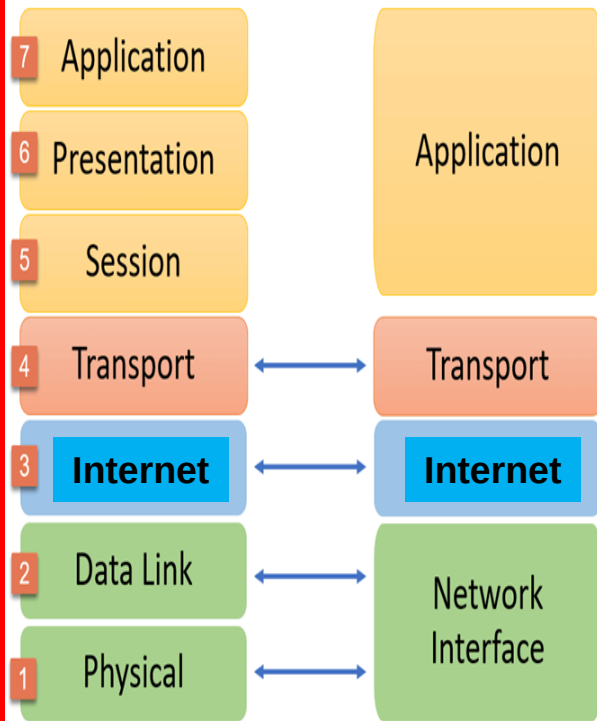
❑ TCP/IP PROTOCOL SUITE

The TCP/IP protocol suite maps to a four-layer conceptual model known as the DARPA model, which was named after the U.S. government agency - Defense Advanced Research Projects Agency (DARPA) that initially developed TCP/IP.

The four layers of the DARPA model are: Application, Transport, Internet, and Network Interface.

OSI Reference Model

TCP/IP Conceptual Layers



□ IP ADDRESS

- An Internet Protocol address (IP address) is a numerical label assigned to each device connected to a computer network that uses the Internet Protocol for communication.
- An IP address serves two principal functions:
 - host or network interface identification and
 - location addressing.

□ IP ADDRESS

❖ IPv4

- Version 4 of the Internet Protocol (IPv4) defines an IP address as a 32-bit number.

❖ IPv6

- However, because of the growth of the Internet and the depletion of available IPv4 addresses, a new version of IP (IPv6), using 128 bits for the IP address, was developed in 1995, and standardized as RFC 2460 in 1998.
- 



□ IP ADDRESS

- IPv6 deployment has been ongoing since the mid-2000s.
- IP addresses are usually written and displayed in human-readable notations, such as

Dotted-decimal notation

172.16.254.1 in IPv4

Colon hex notation

2001:db8:0:1234:0:567:8:1 in IPv6.



□ IP ADDRESS

- The IP address space is managed globally by the Internet Assigned Numbers Authority (IANA)
- Five regional Internet registries (RIR) are responsible in their designated territories for assignment to end users and local Internet registries, such as Internet service providers (ISPs).

□ IP ADDRESS

- IPv4 addresses have been distributed by IANA to the RIRs in blocks of approximately 16.8 million addresses each.
- Each ISP or private network administrator assigns an IP address to each device connected to its network.
- Such assignments may be on a *static* (fixed or permanent) or *dynamic* basis, depending on its software and practices.

SOCKET PROGRAMMING

□ SOCKET CREATION :

- When a socket is created, the program has to specify the *address domain* and the *socket type*.
- Two processes can communicate with each other only if their sockets are of the same type and in the same domain.

SOCKET PROGRAMMING

❑ SOCKET CREATION :

ADDRESS DOMAIN :

There are two widely used address domains :

- UNIX domain
- INTERNET domain
- The *unix domain*, in which two processes which share a common file system communicate, and
- The *Internet domain*, in which two processes running on any two hosts on the Internet communicate.

SOCKET PROGRAMMING

❑ SOCKET CREATION :

ADDRESS FORMAT :

- Each of the address domains has its own address format.
- The address of a socket in the Unix domain is a character string which is basically an entry in the file system.
- The address of a socket in the Internet domain consists of the Internet address of the host machine (every computer on the Internet has a unique 32 bit address, often referred to as its IP address).

SOCKET PROGRAMMING

❑ SOCKET CREATION :

❖ SOCKET TYPES :

- There are two widely used socket types, *stream sockets*, and *datagram sockets*.
- Stream sockets treat communications as a continuous stream of characters, while datagram sockets have to read entire messages at once.
- Each uses its own communications protocol.
- Stream sockets use TCP (Transmission Control Protocol), which is a reliable, stream oriented protocol (connection-oriented)
- Datagram sockets use UDP (User Datagram Protocol), which is unreliable and message oriented. (connection less)
- In our example , we use sockets in the Internet domain using the TCP protocol.

SOCKET PROGRAMMING

❑ SOCKET CREATION :

❖ PORT NUMBER :

- **Each socket needs a port number on that host.**
- **Port numbers are 16 bit unsigned integers.**
- **The lower numbers are reserved in Unix for standard services.**
- **For example, the port number for the FTP server is 21.**
- **It is important that standard services be at the same port on all computers so that clients will know their addresses.**
- **However, port numbers above 2000 upto to 65535 are generally available.**

SOCKET PROGRAMMING

□ DEFAULT TCP PORT NUMBERS

- Port numbers 0 to 1023 are reserved for privileged services and designated as well-known ports.
- This list of port numbers are specified in RFC 1700.
- In TCP/IP and UDP networks, a port is an endpoint to a logical connection and the way a client program specifies a specific server program on a computer in a network.
- The port number identifies what type of port it is.
- For example, port 80 is used for HTTP traffic.

SOCKET PROGRAMMING

□ Well-Known Ports

Some ports have numbers that are assigned to them by the IANA, and these are called the "*well-known ports*" which are specified in RFC 1700.

Port numbers range from 0 to 65535, but only port numbers 0 to 1023 are reserved for privileged services and designated as well-known ports.

SOCKET PROGRAMMING

The following list of *well-known port numbers* specifies the port used by the server process as its contact port

Port Number ---	Description
1	TCP Port Service Multiplexer (TCPMUX)
5	Remote Job Entry (RJE)
7	ECHO
18	Message Send Protocol (MSP)
20	FTP -- Data
21	FTP -- Control
22	SSH Remote Login Protocol
23	Telnet
25	Simple Mail Transfer Protocol (SMTP)
29	MSG ICP
37	Time
42	Host Name Server (Nameserv)
43	Whols
49	Login Host Protocol (Login)
53	Domain Name System (DNS)
69	Trivial File Transfer Protocol (TFTP)
70	Gopher Services
79	Finger
80	HTTP
103	X.400 Standard
108	SNA Gateway Access Server
109	POP2
110	POP3
115	Simple File Transfer Protocol (SFTP)
118	SQL Services

SOCKET PROGRAMMING

Port Number ---	Description
119	Newsgroup (NNTP)
137	NetBIOS Name Service
139	NetBIOS Datagram Service
143	Interim Mail Access Protocol (IMAP)
150	NetBIOS Session Service
156	SQL Server
161	SNMP
179	Border Gateway Protocol (BGP)
190	Gateway Access Control Protocol (GACP)
194	Internet Relay Chat (IRC)
197	Directory Location Service (DLS)
389	Lightweight Directory Access Protocol (LDAP)
396	Novell Netware over IP
443	HTTPS
444	Simple Network Paging Protocol (SNPP)
445	Microsoft-DS
458	Apple QuickTime
546	DHCP Client
547	DHCP Server
563	SNEWS
569	MSN
1080	Socks

- Well-known ports range from 0 through 1023.
- Registered ports are 1024 to 49151.
- Dynamic ports (also called private ports) are 49152 to 65535.

SOCKET PROGRAMMING

❑ STAGES FOR SERVER

The steps involved in establishing a socket on the *server* side are as follows:

1. Socket Creation

Create a socket with the `socket()` system call

2. Set Socket option

Set socket option using `setsockopt()` system call

3. Bind

Bind the socket to an address using the `bind()` system call. For a server socket on the Internet, an address consists of a port number on the host machine.

4. Listen

Listen for connections with the `listen()` system call

5. Accept

Accept a connection with the `accept()` system call. This call typically blocks until a client connects with the server.

6. Send/Recv

Send and receive data

SOCKET PROGRAMMING

❑ STAGES FOR CLIENT

The steps involved in establishing a socket on the *client* side are as follows:

1. Socket Creation

Create a socket with the `socket()` system call

2. Connect

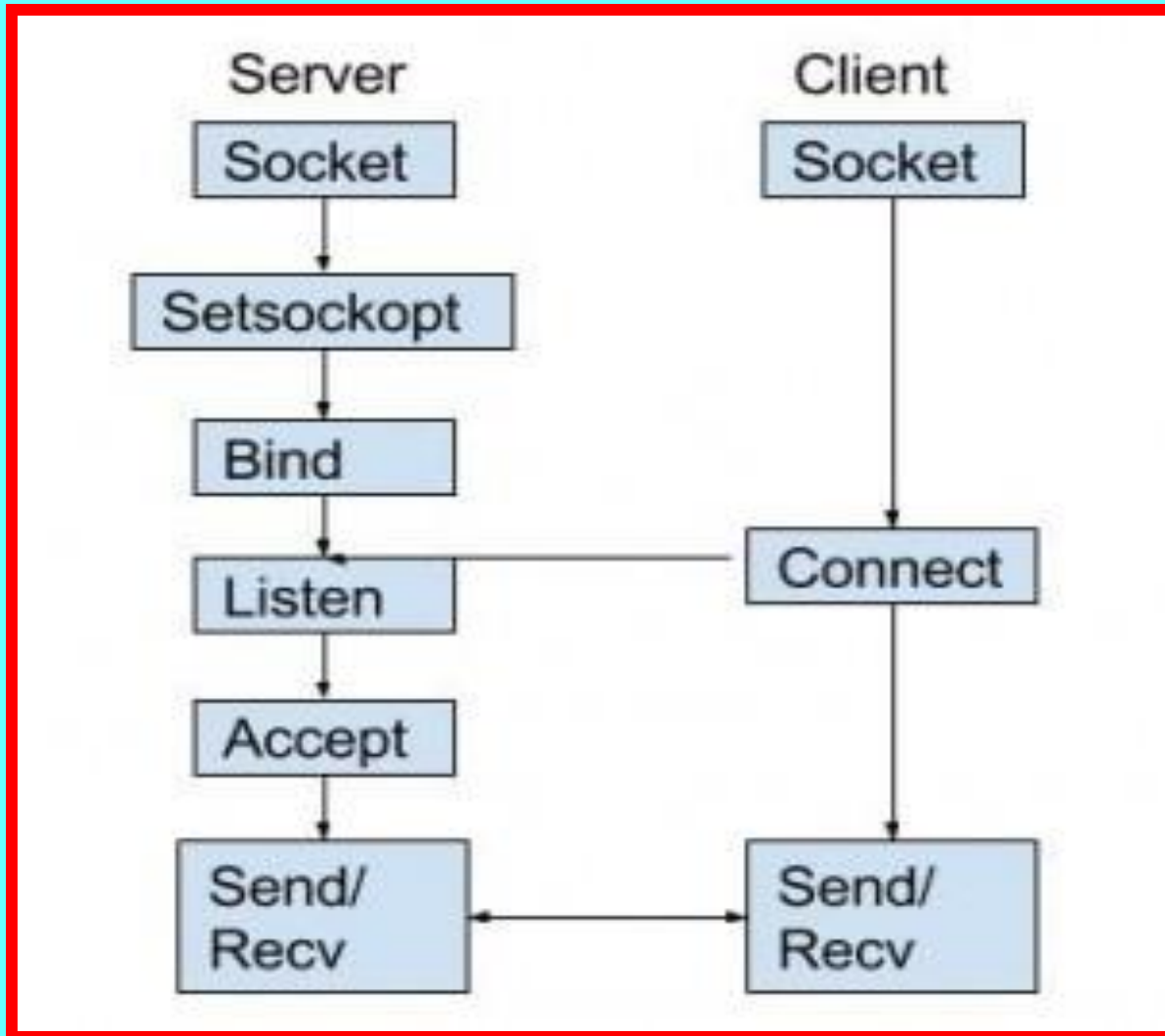
Connect the socket to the address of the server using the `connect()` system call

3. Send/Recv

Send and receive data. There are a number of ways to do this, but the simplest is to use the `read()` and `write()` system calls.

SOCKET PROGRAMMING

State diagram for server and client model



SOCKET PROGRAMMING

❑ Socket creation:

int sockfd = socket(domain, type, protocol)

sockfd: **socket descriptor, an integer (like a file-handle)**

domain: **integer, communication domain**

e.g., AF_INET (IPv4 protocol) , AF_INET6 (IPv6 protocol)

type: **communication type**

SOCK_STREAM: TCP(reliable, connection oriented)

SOCK_DGRAM: UDP(unreliable, connectionless)

protocol: **Protocol value for Internet Protocol(IP), which is 0.**
This is the same number which appears on protocol field in the IP header of a packet.

Example : **server_fd= socket(AF_INET, SOCK_STREAM,0);**

SOCKET PROGRAMMING

❑ Socket creation:

At the beginning, a socket function needs to be declared to get the socket descriptor.

Domain	AF_UNIX - connect inside same machine AF_INET – connect with different machine
Type	SOCK_STREAM – TCP connection SOCK_DGRAM – UDP connection
Protocol	Define here when there is any additional protocol. Otherwise, define it as 0 (IP PROTOCOL) or IPPROTO_IP

Next, decide which `struct` needs to be used based on what domain is used above

AF_UNIX	AF_INET
<pre>struct sockaddr_un { sa_family_t sun_family ; char sun_path[]; };</pre>	<pre>struct sockaddr_in { short int sin_family ; int sin_port; struct in_addr sin_addr; };</pre>
Use struct <code>sockaddr_un</code> if you are using AF_UNIX on your domain. It is required to include <code><sys/un.h></code>	Use struct <code>sockaddr_in</code> if you are using AF_INET on your domain.

sockaddr_in

<code>serv_addr.sin_family = AF_INET;</code>	Define the domain used
<code>serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);</code>	Permit any incoming IP address by declaring INADDR_ANY
<code>serv_addr.sin_port = htons(5000);</code>	Declare port 5000 to be used.

SOCKET PROGRAMMING

❑ Set or get Socket options:

setsockopt(), getsockopt()

Set or Get various options for a socket

Prototypes

#include <sys/types.h>

#include <sys/socket.h>

int getsockopt(int s, int level, int optname, void *optval, socklen_t *optlen);

int setsockopt(int s, int level, int optname, const void *optval, socklen_t optlen);

These functions get and set certain options on a socket and return 0 on success & -1 on error.

Regarding parameters, s is the socket id, level should be set to SOL_SOCKET. Then set the optname to one of the following depending on the requirement :

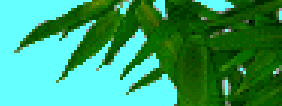
SO_BINDTODEVICE	Bind this socket to a symbolic device name like eth0 instead of using bind() to bind it to an IP address. Type the command ifconfig under Unix to see the device names.
SO_REUSEADDR	Allows other sockets to bind() to this port, unless there is an active listening socket bound to the port already. This enables you to get around those "Address already in use" error messages when you try to restart your server after a crash.
SO_BROADCAST	Allows UDP datagram (SOCK_DGRAM) sockets to send and receive packets sent to and from the broadcast address. Does nothing—NOTHING!!—to TCP stream sockets

The parameter *optval* is usually a pointer to an int indicating the value - for booleans, zero(0) is false, and non-zero(1) is true. If there is no parameter to be passed, *optval* can be NULL. The final parameter, *optlen*, should be set to the length of *optval*, sizeof(int), but varies depending on the option.

Note that in the case of getsockopt(), this is a pointer to a socklen_t, and it specifies the maximum size object that will be stored in *optval* (to prevent buffer overflows).

And getsockopt() will modify the value of *optlen* to reflect the number of bytes actually set.

SOCKET PROGRAMMING



❑ Set or get Socket options- Example

```
int optval;  
int optlen;  
char *optval2;
```

```
// set SO_REUSEADDR on a socket to true (1):
```

```
optval = 1;
```

```
setsockopt(s1, SOL_SOCKET, SO_REUSEADDR, &optval, sizeof optval);
```

```
// bind a socket to a device name (might not work on all systems):
```

```
optval2 = "eth1"; // 4 bytes long, so 4, below:
```

```
setsockopt(s2, SOL_SOCKET, SO_BINDTODEVICE, optval2, 4);
```

```
// see if the SO_BROADCAST flag is set:
```

```
getsockopt(s3, SOL_SOCKET, SO_BROADCAST, &optval, &optlen);
```

```
if (optval != 0)
```

```
{
```

```
    print("SO_BROADCAST enabled on s3!\n");
```

```
}
```

In our example code, we use (opt=1)

```
setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR|SO_REUSEPORT, &opt, sizeof(opt))
```



SOCKET PROGRAMMING



❑ Bind:

`int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);`

After creation of the socket, bind function binds the socket to the address and port number specified in `sockaddr_in` (custom data structure).

`struct sockaddr_in address;`

A `sockaddr_in` is a structure containing an internet address. This structure is defined in `<netinet/in.h>`. Here is the definition:

```
struct sockaddr_in {  
    short  sin_family;  
    u_short sin_port;  
    struct in_addr sin_addr;  
    char   sin_zero[8];  
};
```

An `in_addr` structure, defined in the same header file, contains only one field, a unsigned long called `s_addr`.

```
address.sin_family = AF_INET;  
address.sin_addr.s_addr = INADDR_ANY;  
address.sin_port = htons( PORT );
```

In the example code, we bind the server to the localhost(127.0.0.1), hence we use `INADDR_ANY` to specify the IP address.

```
bind(server_fd, (struct sockaddr *)&address, sizeof(address))
```

SOCKET PROGRAMMING

❑ Bind:

```
int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

After all setup on struct sockaddr_in is done, declare bind function.

Bind function Parameters

**server_socket &
client_socket
sockfd**

Put socket description retrieved on the top

Address addr

Put struct sockaddr_in into it as domain is AF_INET. If your domain is AF_UNIX, try and put struct sockaddr_un here.

addrlen

Put the length of the address

Server and client will start interacting with each other after the `bind` function and it is the most important session.

SOCKET PROGRAMMING

❑ Listen:

int listen(int sockfd, int backlog);

- The listen system call allows the process to listen on the socket for connections.
- The first argument is the socket file descriptor, and the second is the size of the backlog queue, i.e., the number of connections that can be waiting while the process is handling a particular connection.
- This should be set to 5, the maximum size permitted by most systems.
- It puts the server socket in a passive mode, where it waits for the client to approach the server to make a connection.
- The backlog, defines the maximum length to which the queue of pending connections for sockfd may grow.
- If a connection request arrives when the queue is full, the client may receive an error with an indication of ECONNREFUSED.

Example : **listen(server_fd, 5)**

SOCKET PROGRAMMING

❑ Listen:

```
int listen(int sockfd, int backlog);
```

Listen function acts as a waiting room, asking the traffic wait on the waiting room.

Listen function Parameters

server_socket sockfd	Put socket description retrieved on the top
backlog	Define the maximum of awaiting request

SOCKET PROGRAMMING

❑ Accept:

```
int new_socket= accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
```

- The `accept()` system call causes the process to block until a client connects to the server.
- Thus, it wakes up the process when a connection from a client has been successfully established.
- It returns a new file descriptor, and all communication on this connection should be done using the new file descriptor.
- The second argument is a reference pointer to the address of the client on the other end of the connection, and the third argument is the size of this structure.
- It extracts the first connection request on the queue of pending connections for the listening socket, `sockfd`, creates a new connected socket, and returns a new file descriptor referring to that socket.
- At this point, connection is established between client and server, and they are ready to transfer data.

SOCKET PROGRAMMING

❑ Accept:

```
int new_socket= accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
```

Accept function acts as the person who is asking the traffic waiting inside the waiting room to be ready for the meeting between server.

Accept function Parameters

server_socket sockfd	Put socket description retrieved on the top
client_address addr	Client address, Put null here if there is no special request to specify address.
addrlen	Address length, Put null here if second parameter is null
Function returns	Return information of client socket description. Use it for interaction between client and server.

SOCKET PROGRAMMING

❑ Read and Write

- Finally, after the request is accepted, **what should server and client do is read and write data.**
- It is the most simple part in this entire process. read function is used to read the buffer data and write function is used to send the data.

read(sockfd, recvBuff, sizeof(recvBuff)-1)

Read function Parameters

socket_description sockfd	Put server or client socket description depending on reading data from server or client
read buffer recvBuff	Content of the data retrieved
read buffer length Sizeof(recvBuff)-1	Length of the output string

write(sockfd, sendBuff, sizeof(sendBuff)-1)

Write function Parameters

socket_description sockfd	Put server or client socket description depending on sending data to server or client
write buffer sendBuff	Data to be sent
write buffer length Sizeof(sendBuff)-1	Length of the output string

SOCKET PROGRAMMING

❑ Send and Recv

- Finally, after the request is accepted, **what should server and client do is send and receive data.**
- It is the most simple part in this entire process. recv function is used to receive the buffer data and send function is used to send the data.

```
recv(socket_desc , recvBuff , strlen(recvBuff), 0)
```

recv function Parameters

socket_description sockfd	Put server or client socket description depending on reading data from server or client
recv buffer recvBuff	Content of the data retrieved
recv buffer length Sizeof(recvBuff)	Length of the output string

```
send(socket_desc , sendBuff , strlen(sendBuff) , 0)
```

send function Parameters

socket_description sockfd	Put server or client socket description depending on sending data to server or client
send buffer sendBuff	Data to be sent
send buffer length Sizeof(sendBuff)	Length of the output string

SOCKET PROGRAMMING

❑ Close Socket

- **Function close** is used to close the socket.
- **Need to include the unistd.h header file for this.**

```
close(socket_desc);
```

close function Parameter

socket_description Socket_desc	Put server or client socket description
-----------------------------------	---

Example :
Close(sockfd)

SOCKET PROGRAMMING

❑ Program Name : server.c

// Server side C program to demonstrate Socket programming

```
#include <stdio.h>
```

```
#include <sys/socket.h>
```

```
#include <sys/types.h>
```

```
#include <sys/stat.h>
```

```
#include <fcntl.h>
```

```
#include <unistd.h>
```

```
#include <stdlib.h>
```

```
#include <netinet/in.h>
```

```
#include <string.h>
```

```
#define PORT 8080
```

```
int main(int argc, char const *argv[])
```

```
{
```

```
    int server_fd, new_socket, valread;
```

```
    struct sockaddr_in address;
```

```
    int opt = 1;
```

```
    int addrlen = sizeof(address);
```

```
    char buffer[1024] = {0}, schat[1024];
```

```
    char *hello = "Hello from server";
```

```
    char *bye = "bye";
```

SOCKET PROGRAMMING

❑ Program Name : server.c (continued)

```
printf("CREATING SERVER SOCKET .....\\n");  
// Creating socket file descriptor  
if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0)  
{  
    perror("socket creation failed");  
    exit(EXIT_FAILURE);  
}
```

```
printf("DEFINING SERVER SOCKET OPTIONS, FAMILY, ADDRESS & PORT .....\\n");  
if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR | SO_REUSEPORT, &opt, sizeof(opt)))  
{  
    perror("setsockopt failed");  
    exit(EXIT_FAILURE);  
}
```

```
address.sin_family = AF_INET;  
address.sin_addr.s_addr = INADDR_ANY;  
address.sin_port = htons( PORT );
```

SOCKET PROGRAMMING



❑ Program Name : server.c (continued)

```
printf("BINDING SERVER SOCKET TO PORT 8080 .....\\n");
// Forcefully attaching socket to the port 8080
if (bind(server_fd, (struct sockaddr *)&address, sizeof(address))<0)
{
    perror("socket bind failed");
    exit(EXIT_FAILURE);
}

printf("SERVER LISTENING ON PORT 8080 FOR NEW CONNECTION.....\\n");
if (listen(server_fd, 3) < 0)
{
    perror("socket listen failed ");
    exit(EXIT_FAILURE);
}

if ((new_socket=accept(server_fd, (struct sockaddr *)&address, (socklen_t*)&addrlen))<0)
{
    perror("socket accept failed");
    exit(EXIT_FAILURE);
}

printf("SERVER ACCEPTED NEW CONNECTION FROM A CLIENT ON PORT 8080..\\n");
```



SOCKET PROGRAMMING

❑ Program Name : server.c (continued)

```
valread = read( new_socket , buffer, 1024);
printf("%s\n",buffer );
send(new_socket , hello , strlen(hello) , 0 );
printf(" Ready for Chat....\n");

while(1)
{
    memset(buffer, 0, sizeof(buffer));
    valread = read( new_socket , buffer, 1024);
    printf("%s\n",buffer );
    memset(schat, 0, sizeof(schat));
    printf("server : ");
    fgets (schat, sizeof(schat), stdin);
    send(new_socket , schat , strlen(schat) , 0 );
    schat[strlen(schat)] = '\0';
    if(strncmp(schat , bye, strlen(bye))==0)
        break;
}
return 0;
}
```

SOCKET PROGRAMMING

❑ CLIENT SIDE

- CREATE CLIENT SOCKET

sock = socket(AF_INET, SOCK_STREAM, 0)

- CONNECT

int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);

- **The connect function is called by the client to establish a connection to the server.**
- **It takes three arguments, the socket file descriptor, the address of the host to which it wants to connect (including the port number), and the size of this address. This function returns 0 on success and -1 if it fails.**
- The connect() system call connects the socket referred to by the file descriptor sockfd to the address specified by addr.
- Server's address and port is specified in addr.
- **Notice that the client needs to know the port number of the server, but it does not need to know its own port number. This is typically assigned by the system when connect is called.**

SOCKET PROGRAMMING

❑ CONNECT

```
int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

Lastly, connect function acts as the person who wants to carry out some work with the server.

Connect function Parameters

client_socket sockfd	Put socket description retrieved on the top
address addr	Put the struct sockaddr defined on the top
addrlen	Put the length of the address

SOCKET PROGRAMMING

❑ Program Name : client.c

// Client side C program to demonstrate Socket programming

```
#include <stdio.h>
```

```
#include <sys/socket.h>
```

```
#include <sys/types.h>
```

```
#include <sys/stat.h>
```

```
#include <fcntl.h>
```

```
#include <unistd.h>
```

```
#include <stdlib.h>
```

```
#include <netinet/in.h>
```

```
#include <string.h>
```

```
#include <arpa/inet.h>
```

```
#define PORT 8080
```

```
int main(int argc, char const *argv[])
```

```
{
```

```
    struct sockaddr_in address;
```

```
    int sock = 0, valread;
```

```
    struct sockaddr_in serv_addr;
```

```
    char *hello = "Hello from client";
```

```
    char buffer[1024] = {0}, cchat[1024];
```

```
    char *bye = "bye";
```

SOCKET PROGRAMMING

❑ Program Name : client.c (continued)

```
printf("CREATING CLIENT SOCKET .....\\n");
if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0)
{
    printf("\\n Socket creation error \\n");
    return -1;
}
printf("DEFINING SOCKET FAMILY, ADDRESS & PORT .....\\n");
memset(&serv_addr, '0', sizeof(serv_addr));
serv_addr.sin_family = AF_INET;
serv_addr.sin_port = htons(PORT);
// Convert IPv4 and IPv6 addresses from text to binary form
if(inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr)<=0)
{
    printf("\\nInvalid address/ Address not supported \\n");
    return -1;
}
printf("CLIENT CONNECTING ON PORT 8080 TO COMMUNICATE WITH SERVER..\\n");
if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0)
{
    printf("\\nConnection Failed \\n");
    return -1;
}
```

SOCKET PROGRAMMING

❑ Program Name : client.c (continued)

```
send(sock , hello , strlen(hello) , 0 );
printf("Hello message sent\n");
valread = read( sock , buffer, 1024);
printf("%s\n",buffer );
printf(" Ready for Chat....\n");

while(1)
{
    memset(buffer, 0, sizeof(buffer));
    memset(cchat, 0, sizeof(cchat));
    printf("CLIENT : ");
    fgets (cchat, sizeof(cchat), stdin);
    send(sock , cchat , strlen(cchat) , 0 );
    valread = read( sock , buffer, 1024);
    printf("%s\n",buffer );
    cchat[strlen(cchat)] = '\0';
    if(strncmp(cchat, bye, strlen(bye))==0)
        break;
}
return 0;
}
```

Compile the files server.c and client.c
Run(Setup) the server on one xterm by using
\$./server
After that Run(Setup) the client on another xterm by using
\$./client
Then initiate a chat session between them

SOC 3010

OPERATING SYSTEM

SOCKET PROGRAMMING

READING ASSIGNMENTS

SOCKET PROGRAMMING

❑ Program Name : gethostname.c

```
/* When connecting to a remote host , it is necessary to have its IP address. */
/* Function gethostbyname is used for this purpose. */
/* It takes the domain name as the parameter and returns a structure of type hostent. */
/* This structure has the ip information. */
/* It is present in netdb.h. Lets have a look at this structure */
/* Description of data base entry for a single host. */
```

```
/* struct hostent */
/* {
/*     char *h_name;           Official name of host.
/*     char **h_aliases;       Alias list.
/*     int  h_addrtype;         Host address type.
/*     int  h_length;           Length of address.
/*     char **h_addr_list;      List of addresses from name server.
/* };
/*     The h_addr_list has the IP addresses. */
```

```
#include<stdio.h>           //printf
#include<string.h>           //strcpy
#include<sys/socket.h>
#include<netdb.h>            //hostent
#include<arpa/inet.h>
```

```
int main(int argc , char *argv[])
{
    char *hostname;
    char ip[100];
    struct hostent *he;
    struct in_addr **addr_list;
    int i;
```

SOCKET PROGRAMMING

❑ Program Name : gethostname.c

```
if(argc == 2 )
```

```
{
    hostname = argv[1];
    if ( (he = gethostbyname( hostname ) ) == NULL)
    {
        //gethostbyname failed
        perror("gethostbyname");
        return 1;
    }
```

```
//Cast the h_addr_list to in_addr , since h_addr_list also has the ip address in long format only
addr_list = (struct in_addr **) he->h_addr_list;
```

```
for(i = 0; addr_list[i] != NULL; i++)
{
    //Return the first one;
    strcpy(ip , inet_ntoa(*addr_list[i]) );
    printf("ip = %s \n", ip);
}
```

```
printf("%s resolved to : %s \n" , hostname , ip);
}
```

```
else
```

```
    printf("Usage : ./gethostname domain_name \n");
    return 0;
```

```
}
```

```
// compile using gcc -o gethostname gethostname.c
```

```
// execute using ./gethostname domain_name - for example : $./gethostname www.google.com
```

SOCKET PROGRAMMING



❑ Conversion Functions :

```
#include <sys/socket.h>
```

```
#include <netinet/in.h>
```

```
#include <arpa/inet.h>
```

1. inet_aton

```
int inet_aton(const char *cp, struct in_addr *inp);
```

- **inet_aton()** converts the Internet host address *cp* from the IPv4 numbers-and-dots notation into binary form (in network byte order) and stores it in the structure that *inp* points to.
- **inet_aton()** returns nonzero if the address is valid, zero if not.

The address supplied in *cp* can have one of the following forms:

- a.b.c.d* - Each of the four numeric parts specifies a byte of the address; the bytes are assigned in left-to-right order to produce the binary address.
- a.b.c* - Parts *a* and *b* specify the first two bytes of the binary address. Part *c* is interpreted as a 16-bit value that defines the rightmost two bytes of the binary address. This notation is suitable for specifying (outmoded) Class B network addresses.
- a.b* - Part *a* specifies the first byte of the binary address. Part *b* is interpreted as a 24-bit value that defines the rightmost three bytes of the binary address. This notation is suitable for specifying (outmoded) Class C network addresses.
- a* - The value *a* is interpreted as a 32-bit value that is stored directly into the binary address without any byte rearrangement.

In all of the above forms, components of the dotted address can be specified in decimal, octal (with a leading 0), or hexadecimal, with a leading 0x). Addresses in any of these forms are collectively termed *IPv4 numbers-and-dots notation*. **The form that uses exactly four decimal numbers is referred to as *IPv4 dotted-decimal notation* (or sometimes: *IPv4 dotted-quad notation*).**

SOCKET PROGRAMMING

❑ Conversion Functions :

2. `inet_addr()`

`in_addr_t inet_addr(const char *cp);`

- The **`inet_addr()`** function converts the Internet host address *cp* from IPv4 numbers-and-dots notation into binary data in network byte order.
- If the input is invalid, **`INADDR_NONE`** (usually -1) is returned.
- Use of this function is problematic because -1 is a valid address (255.255.255.255).

3. `inet_network()`

`in_addr_t inet_network(const char *cp);`

- The **`inet_network()`** function converts *cp*, a string in IPv4 numbers-and-dots notation, into a number in host byte order suitable for use as an Internet network address.
- On success, the converted address is returned. If the input is invalid, -1 is returned.

SOCKET PROGRAMMING

❑ Conversion Functions :

4. `inet_ntoa()`

`char *inet_ntoa(struct in_addr in);`

- The `inet_ntoa()` function converts the Internet host address *in*, given in network byte order, to a string in IPv4 dotted-decimal notation.
- The string is returned in a statically allocated buffer, which subsequent calls will overwrite.

5. `inet_lnaof()`

`in_addr_t inet_lnaof(struct in_addr in);`

- The `inet_lnaof()` function returns the local network address part of the Internet address *in*. The returned value is in host byte order.

6. `inet_netof()`

`in_addr_t inet_netof(struct in_addr in);`

- The `inet_netof()` function returns the network number part of the Internet address *in*.
- The returned value is in host byte order.

SOCKET PROGRAMMING

❑ Conversion Functions :

7. `inet_makeaddr()`

```
struct in_addr  inet_makeaddr(int net, int host);
```

- The `inet_makeaddr()` function is the converse of `inet_netof()` and `inet_lnaof()`.
- It returns an Internet host address in network byte order, created by combining the network number *net* with the local address *host*, both in host byte order.

The structure `in_addr` as used in `inet_ntoa()`, `inet_makeaddr()`, `inet_lnaof()` and `inet_netof()` is defined in `<netinet/in.h>` as:

```
typedef uint32_t  in_addr_t;
```

```
struct in_addr {  
    in_addr_t  s_addr;  
};
```

SOCKET PROGRAMMING

❑ Conversion Functions :

```
#define _BSD_SOURCE
#include <arpa/inet.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    struct in_addr addr;

    if (argc != 2) {
        fprintf(stderr, "%s <dotted-address>\n", argv[0]);
        exit(EXIT_FAILURE);
    }

    if (inet_aton(argv[1], &addr) == 0) {
        fprintf(stderr, "Invalid address\n");
        exit(EXIT_FAILURE);
    }

    printf("%s\n", inet_ntoa(addr));
    exit(EXIT_SUCCESS);
}
```

SOCKET PROGRAMMING

❑ Handling multiple clients on server without multi threading

Select()

- A better way to handle multiple clients is by using **select()** linux command.
- Select command allows to monitor multiple file descriptors, waiting until one of the file descriptors become active.
- For example, if there is some data to be read on one of the sockets select will provide that information.
- **Select** works like an interrupt handler, which gets activated as soon as any file descriptor sends any data.

Data structure used for select - fd_set

- It contains the list of file descriptors to monitor for some activity.
There are four functions associated with fd_set: **fd_set readfds;**
 - **// Clear an fd_set**
 - **FD_ZERO(&readfds);**
 - **// Add a descriptor to an fd_set**
 - **FD_SET(master_sock, &readfds);**
 - **// Remove a descriptor from an fd_set**
 - **FD_CLR(master_sock, &readfds);**
 - **//If something happened on the master socket , then its an incoming connection**
 - **FD_ISSET(master_sock, &readfds);**
- **Activating select:**
activity = select(max_fd + 1 , &readfds , NULL , NULL , NULL);

SOCKET PROGRAMMING

❑ Handling multiple clients on server without multi threading

Select() - Check if sockets descriptors are ready to read/write

```
#include <sys/select.h>
```

```
int select(int n, fd_set *readfds, fd_set *writefds, fd_set *exceptfds, struct timeval *timeout);
```

```
FD_SET(int fd, fd_set *set);
```

```
FD_CLR(int fd, fd_set *set);
```

```
FD_ISSET(int fd, fd_set *set);
```

```
FD_ZERO(fd_set *set);
```

Returns the number of descriptors in the set on success, 0 if the timeout was reached, or -1 on error

Description

The **select()** function gives you a way to simultaneously check multiple sockets to see if they have data waiting to be **recv()**d, or if you can **send()** data to them without blocking, or if some exception has occurred.

Populate your sets of socket descriptors using the macros, like **FD_SET()**, above. Once you have the set, you pass it into the function as one of the following parameters: *readfds* if you want to know when any of the sockets in the set is ready to **recv()** data, *writefds* if any of the sockets is ready to **send()** data to, and/or *exceptfds* if you need to know when an exception (error) occurs on any of the sockets. Any or all of these parameters can be **NULL** if you're not interested in those types of events. After **select()** returns, the values in the sets will be changed to show which are ready for reading or writing, and which have exceptions.

- The first parameter, *n* is the highest-numbered socket descriptor plus one.
- Lastly, the struct timeval, *timeout*, at the end to tell **select()** how long to check these sets for. It'll return after the timeout, or when an event occurs, whichever is first. The struct timeval has two fields: *tv_sec* is the number of seconds, to which is added *tv_usec*, the number of microseconds (1,000,000 microseconds in a second.)
- The helper macros do the following:

FD_SET(int fd, fd_set *set);

Add fd to the set.

FD_CLR(int fd, fd_set *set);

Remove fd from the set.

FD_ISSET(int fd, fd_set *set);

Return true if fd is in the set.

FD_ZERO(fd_set *set);

Clear all entries from the set.

SOCKET PROGRAMMING

❑ Handling multiple clients on server without multi threading

//Example code: A simple server side code, which echos back the received message. **filename : servernmt.c**

//Handle multiple socket connections with select and fd_set on Linux

```
#include <stdio.h>
#include <string.h> //strlen
#include <stdlib.h>
#include <errno.h>
#include <unistd.h> //close
#include <arpa/inet.h> //close
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <sys/time.h> //FD_SET, FD_ISSET, FD_ZERO macros
#define TRUE 1
#define FALSE 0
#define PORT 8888
int main(int argc , char *argv[])
{
    int opt = TRUE;
    int master_socket , addrlen , new_socket , client_socket[30] , max_clients = 30 , activity, i , valread , sd;
    int max_sd;
    struct sockaddr_in address;
    char buffer[1025]; //data buffer of 1K

    //set of socket descriptors
    fd_set readfds;
    //a message
    char *message = "ECHO Daemon v1.0 \r\n";
    //initialise all client_socket[] to 0 so not checked
    for (i = 0; i < max_clients; i++)
    {
        client_socket[i] = 0;
    }
    //create a master socket
    if( (master_socket = socket(AF_INET , SOCK_STREAM , 0)) == 0)
    {
        perror("socket creation failed");
        exit(EXIT_FAILURE);
    }
```

SOCKET PROGRAMMING

❑ Handling multiple clients on server without multi threading

```
//set master socket to allow multiple connections ,
//this is just a good habit, it will work without this
if( setsockopt(master_socket, SOL_SOCKET, SO_REUSEADDR, (char *)&opt, sizeof(opt)) < 0 )
{
    perror("setsockopt");
    exit(EXIT_FAILURE);
}
//type of socket created
address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY;
address.sin_port = htons( PORT );

//bind the socket to localhost port 8888
if (bind(master_socket, (struct sockaddr *)&address, sizeof(address))<0)
{
    perror("bind failed");
    exit(EXIT_FAILURE);
}
printf("Listener on port %d \n", PORT);
//try to specify maximum of 3 pending connections for the master socket
if (listen(master_socket, 3) < 0)
{
    perror("listen");
    exit(EXIT_FAILURE);
}
//accept the incoming connection
addrlen = sizeof(address);
puts("Waiting for connections ...");
while(TRUE)
{
    //clear the socket set
    FD_ZERO(&readfds);
    //add master socket to set
    FD_SET(master_socket, &readfds);
    max_sd = master_socket;
    //add child sockets to set
    for ( i = 0 ; i < max_clients ; i++)
    {
        //socket descriptor
        sd = client_socket[i];
        //if valid socket descriptor then add to read list
        if(sd > 0)
            FD_SET( sd , &readfds);
        //highest file descriptor number, need it for the select function
        if(sd > max_sd)
            max_sd = sd;
    }
}
```

SOCKET PROGRAMMING

❑ Handling multiple clients on server without multi threading

```
//wait for an activity on one of the sockets , timeout is NULL ,
//so wait indefinitely
activity = select( max_sd + 1 , &readfds , NULL , NULL , NULL);

if ((activity < 0) && (errno!=EINTR))
{
    printf("select error");
}

//If something happened on the master socket ,
//then its an incoming connection
if (FD_ISSET(master_socket, &readfds))
{
    if ((new_socket = accept(master_socket, (struct sockaddr *)&address, (socklen_t*)&addrlen))<0)
    {
        perror("accept");
        exit(EXIT_FAILURE);
    }

    //inform user of socket number - used in send and receive commands
    printf("New connection , socket fd is %d , ip is : %s , port : %d \n" , new_socket , inet_ntoa(address.sin_addr) , ntohs
        (address.sin_port));

    //send new connection greeting message
    if( send(new_socket, message, strlen(message), 0) != strlen(message) )
    {
        perror("send");
    }

    puts("Welcome message sent successfully");

    //add new socket to array of sockets
    for (i = 0; i < max_clients; i++)
    {
        //if position is empty
        if( client_socket[i] == 0 )
        {
            client_socket[i] = new_socket;
            printf("Adding to list of sockets as %d\n" , i);

            break;
        }
    }
}
```


SOCKET PROGRAMMING

❑ Handling multiple clients on server without multi threading

```
//else its some IO operation on some other socket
for (i = 0; i < max_clients; i++)
{
    sd = client_socket[i];

    if (FD_ISSET( sd , &readfds))
    {
        //Check if it was for closing , and also read the
        //incoming message
        if ((valread = read( sd , buffer, 1024)) == 0)
        {
            //Somebody disconnected , get his details and print
            getpeername(sd , (struct sockaddr*)&address , (socklen_t*)&addrlen);
            printf("Host disconnected , ip %s , port %d \n" , inet_ntoa(address.sin_addr) , ntohs(address.sin_port));

            //Close the socket and mark as 0 in list for reuse
            close( sd );
            client_socket[i] = 0;
        }

        //Echo back the message that came in
        else
        {
            //set the string terminating NULL byte on the end
            //of the data read
            buffer[valread] = '\0';
            send(sd , buffer , strlen(buffer) , 0 );
        }
    }
}

return 0;
}
```

Compile the file and run the server.

```
$gcc -o servernmt servernmt.c
```

```
$/servernmt
```

Use telnet to connect the server as a client from other xterms.

Try running on different machines/xterms using following command:

```
telnet localhost 8888
```

SOCKET PROGRAMMING

❑ Handling multiple clients on server with multi threading filename : servermt.c

//Handle multiple socket connections with threads
//To handle every connection we need a separate handling code to run along
//with the main server accepting connections.
//One way to achieve this is using threads.
//The main server program accepts a connection and creates a new thread to handle
//communication for the connection, and then the server goes back to
//accept more connections.
//On Linux threading can be done with the pthread (posix threads) library.
// use threads to create handlers for each connection the server accepts

```
#include<stdio.h>
#include<string.h> //strlen
#include<stdlib.h> //strlen
#include<sys/socket.h>
#include<arpa/inet.h> //inet_addr
#include<unistd.h> //write
#include<pthread.h> //for threading , link with lpthread
void *connection_handler(void *);

int main(int argc , char *argv[])
{
    int socket_desc , new_socket , c , *new_sock;
    struct sockaddr_in server , client;
    char *message;
    //Create socket
    socket_desc = socket(AF_INET , SOCK_STREAM , 0);
    if (socket_desc == -1)
    {
        printf("Could not create socket");
    }

    //Prepare the sockaddr_in structure
    server.sin_family = AF_INET;
    server.sin_addr.s_addr = INADDR_ANY;
    server.sin_port = htons( 9999 );

    //Bind
    if( bind(socket_desc,(struct sockaddr *)&server , sizeof(server)) < 0)
    {
        puts("bind failed");
        return 1;
    }
    puts("bind done");
```

SOCKET PROGRAMMING

❑ Handling multiple clients on server with multi threading

```
//Listen
listen(socket_desc , 3);

//Accept and incoming connection
puts("Waiting for incoming connections...");
c = sizeof(struct sockaddr_in);
while( (new_socket = accept(socket_desc, (struct sockaddr *)&client, (socklen_t*)&c)) )
{
    puts("Connection accepted");

    //Reply to the client
    message = "Hello Client , I have received your connection. And now I will assign a handler for you\n";
    write(new_socket , message , strlen(message));

    pthread_t sniffer_thread;
    new_sock = (int*)malloc(sizeof(int));
    *new_sock = new_socket;

    if( pthread_create( &sniffer_thread , NULL , connection_handler , (void*) new_sock) < 0)
    {
        perror("could not create thread");
        return 1;
    }

    //Now join the thread , so that we dont terminate before the thread
    //pthread_join( sniffer_thread , NULL);
    puts("Handler assigned");
}

if (new_socket<0)
{
    perror("accept failed");
    return 1;
}

return 0;
}
```

SOCKET PROGRAMMING

❑ Handling multiple clients on server with multi threading

/* * This will handle connection for each client * */

void *connection_handler(void *socket_desc)

```
{
    //Get the socket descriptor
    int sock = *(int*)socket_desc;
    int read_size;
    char message[2000] , client_message[2000], schat[2000], *bye, s[10], chat[2000], s1[100];
    bye = "bye";
    //Send some messages to the client
    strcpy(message, "To ");
    sprintf(s, "%d", sock);
    strcat(message, s, strlen(s));
    strcpy(s1, " : Greetings! I am your connection handler\n");
    strcat(message, s1, strlen(s1));
    write(sock , message , strlen(message));
    memset(message, 0, sizeof(message));
    read_size = read(sock , client_message , strlen(client_message));
    printf("From Client [%d] : %s\n", sock, client_message );
    strcpy(message, "To ");
    sprintf(s, "%d", sock);
    strcat(message, s, strlen(s));
    strcpy(s1, "Now we shall have a chat session : so type something \n");
    strcat(message, s1, strlen(s1));
    write(sock , message , strlen(message));
    //Receive a message from client
    while(1)
    {
        memset(client_message, 0, sizeof(client_message));
        read_size = read(sock , client_message , strlen(client_message));
        printf("From Client [%d] : %s\n", sock, client_message );
        memset(schat, 0, sizeof(schat));
        printf("server : ");
        fgets (schat, sizeof(schat), stdin);
        strcpy(chat, "To ");
        sprintf(s, "%d", sock);
        strcat(chat, s, strlen(s));
        strcat(chat, schat, strlen(schat));
        send(sock, chat , strlen(chat) , 0 );
        schat[strlen(schat)] = '\0';
        if(strncmp(schat , bye, strlen(bye))==0)
            break;
    }
    puts("Client disconnected");
    fflush(stdout);
    //Free the socket pointer
    free(socket_desc);
    return 0;
}
```

Compile the file and run the server.

`$gcc -o servernmt servermt.c`

`$/servermt`

Use telnet to connect the server as a client from other xterms.

Try running on different machines/xterms using following command:

`telnet localhost 9999`

SOC 3010

OPERATING SYSTEM

END OF
WEEK 9 LECTURE1

SOCKET
PROGRAMMING

WISH YOU ALL THE BEST

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 3010

OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

